

Ataques adversarios sobre clasificadores convolucionales en CIFAR-10: evasión (FGSM, PGD, C&W) y ataques en entrenamiento (backdoor y envenenamiento)

Andrés Ángel

Universidad Nacional de Colombia

Redes neuronales: arquitecturas y aplicaciones — Prof. Arles Rodríguez

18 de junio de 2026

Resumen

Los modelos de aprendizaje profundo, pese a alcanzar precisión sobrehumana en clasificación de imágenes, presentan vulnerabilidades adversarias sistemáticas. Este trabajo estudia dos grandes familias de amenazas sobre clasificadores convolucionales entrenados en CIFAR-10. En la **Parte I-II** (ataques de *evasión* en inferencia) replicamos FGSM [2], PGD [3] y Carlini–Wagner [4], implementando FGSM y PGD desde cero y verificándolos contra *torchattacks*. Una ResNet-18 con 93.5% de accuracy limpia cae a 15.8% bajo FGSM y a 0% bajo PGD-40 (ℓ_∞ , $\epsilon = 8/255$); C&W alcanza misclassification total con perturbaciones ℓ_2 casi diez veces menores. Estudiamos además la transferibilidad entre arquitecturas y las normas de las perturbaciones. En la **Parte III** (ataques en *entrenamiento*) implementamos un backdoor tipo BadNets [11], label-flipping y clean-label poisoning (Poison Frogs [12]). El backdoor resulta especialmente alarmante: envenenando apenas el 0.5% del dataset se logra un *attack success rate* del $\sim 92\%$ manteniendo la accuracy limpia intacta ($\sim 91\%$), lo que lo hace prácticamente indetectable mediante validación estándar. Los resultados ilustran que la vulnerabilidad adversaria es una propiedad estructural de los clasificadores entrenados de forma convencional, presente tanto en inferencia como en la cadena de suministro de datos.

1 Introducción

Desde el descubrimiento de los *ejemplos adversarios* por Szegedy et al. [1], se ha consolidado la observación de que las redes neuronales profundas son vulnerables a perturbaciones δ de norma ℓ_p minúscula que, sumadas a una imagen x correctamente clasificada, producen una entrada $x' = x + \delta$ que el modelo clasifica incorrectamente. La relevancia práctica de este fenómeno es directa: sistemas de visión por computador desplegados en vehículos autónomos, biometría facial o moderación de contenido pueden ser engañados por perturbaciones imperceptibles para un humano [9].

El estudio de estos ataques no solo expone vulnerabilidades de seguridad sino que ilumina propiedades geométricas profundas de los clasificadores: sugiere que las fronteras de decisión aprendidas, aunque correctas en la *data manifold*, son frágiles en sus direcciones normales [8].

Aporte de este trabajo. Replicamos tres ataques fundamentales (FGSM, PGD, CW) sobre dos arquitecturas (ResNet-18 estándar y un modelo robusto de RobustBench), reportamos curvas accuracy-vs- ϵ y discutimos la relación entre fortaleza del ataque, dualidad de normas y formulación min-max del entrenamiento adversario.

2 Trabajo relacionado

Los ataques adversarios en visión se organizan habitualmente en tres ejes:

Conocimiento del atacante

white-box (acceso a θ y gradientes), *black-box* (solo consultas), o *transfer* (atacar usando un modelo sustituto).

Norma de la perturbación

ℓ_∞ [2, 3], ℓ_2 [4], ℓ_0 [10], o restricciones perceptuales más sofisticadas.

Momento del ataque

evasión en inferencia (objeto de este trabajo) versus *envenenamiento* en entrenamiento o *backdoors* [11].

Las defensas más confiables a la fecha son variantes del *adversarial training* de Madry et al. [3], que formula el entrenamiento como un problema saddle-point. RobustBench [5] estandariza la evaluación con AutoAttack [6], un ensemble de ataques que evita la sobreestimación de robustez documentada en defensas anteriores.

Nuestra implementación se apoya en `torchattacks` [7], librería en PyTorch que provee implementaciones de referencia de los ataques considerados.

3 Dataset y características

Empleamos **CIFAR-10** [14]: 60.000 imágenes RGB de 32×32 píxeles distribuidas en 10 clases balanceadas (avión, automóvil, ave, gato, ciervo, perro, rana, caballo, barco, camión), con 50.000 imágenes para entrenamiento y 10.000 para evaluación.

Por qué este dataset. CIFAR-10 es el benchmark estándar de la literatura de robustez adversaria: ofrece resolución suficiente para que las arquitecturas convolucionales modernas extraigan jerarquías de features no triviales, pero permite entrenar modelos completos en horas con GPU modesta. La gran mayoría de papers sobre defensas y ataques (incluyendo todo el leaderboard de RobustBench en ℓ_∞) reportan métricas en CIFAR-10, lo que permite comparar nuestros resultados con valores de referencia publicados.

Pre-procesamiento. Las imágenes se normalizan a $[0, 1]$ *antes* de aplicar el ataque y la normalización con media/desviación de canal se incorpora como primera capa del modelo. Esto es crítico: la restricción $\|\delta\|_\infty \leq 8/255$ debe interpretarse en el espacio de píxeles original.

4 Métodos

4.1 Formulación del ataque

Sea $f_\theta : [0, 1]^d \rightarrow \mathbb{R}^K$ un clasificador con K clases y predicción $\hat{y}(x) = \arg \max_k f_\theta(x)_k$. Dada una imagen x con etiqueta verdadera y y una pérdida $\mathcal{L}$ (cross-entropy), el problema del atacante es:

$$\delta^* = \arg \max_{\delta \in \Delta} \mathcal{L}(f_\theta(x + \delta), y), \quad \Delta = \{\delta \in \mathbb{R}^d : \|\delta\|_p \leq \epsilon, x + \delta \in [0, 1]^d\}. \quad (1)$$

Se trata de un problema no convexo (por la red) sobre un convexo (intersección de bola ℓ_p y caja unitaria). Para CIFAR-10 fijamos $p = \infty$ y $\epsilon = 8/255$ siguiendo el estándar de RobustBench.

4.2 FGSM (Goodfellow et al., 2014)

Linealizando (1) via Taylor de primer orden y resolviendo exactamente vía dualidad Hölder ($\ell_\infty^* = \ell_1$):

$$\delta^{\text{FGSM}} = \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}(f_\theta(x), y)). \quad (2)$$

La actualización es *one-shot*: el adversario se sitúa en el vértice del hipercubo $\{x' : \|x' - x\|_\infty \leq \epsilon\}$ que más alinea con el gradiente de la pérdida.

4.3 PGD (Madry et al., 2018)

Sin linealizar, se realiza ascenso de gradiente proyectado:

$$\delta^{(t+1)} = \Pi_\Delta \left(\delta^{(t)} + \alpha \cdot \text{sign}(\nabla_x \mathcal{L}(f_\theta(x + \delta^{(t)}), y)) \right), \quad (3)$$

donde Π_Δ es la proyección sobre Δ (clipping coordenada a coordenada para ℓ_∞). Inicializamos $\delta^{(0)} \sim U(\Delta)$ (*random start*) y corremos T iteraciones con paso $\alpha = \epsilon/4$. Madry et al. argumentan empíricamente que PGD con suficientes restarts es el adversario de primer orden más fuerte dentro de Δ .

4.4 Carlini–Wagner (C&W, 2017)

Reformulando como problema de minimización de la perturbación sujeto a misclassification:

$$\min_{\delta} \|\delta\|_2^2 + c \cdot g(x + \delta), \quad g(x') = \max \left(f_\theta(x')_y - \max_{k \neq y} f_\theta(x')_k, -\kappa \right). \quad (4)$$

La función g es un sustituto suave de la indicatriz de error. El parámetro $c > 0$ se busca por bisección. La restricción $x + \delta \in [0, 1]^d$ se impone via reparametrización $x + \delta = \frac{1}{2}(\tanh(w) + 1)$, optimizando sobre $w \in \mathbb{R}^d$ sin restricciones con Adam.

4.5 AutoAttack (Croce & Hein, 2020)

AutoAttack [6] es un *ensemble* de cuatro ataques *parameter-free* que constituye el estándar de evaluación de RobustBench: APGD-CE y APGD-DLR (versiones de PGD con paso adaptativo sobre dos funciones de pérdida distintas), FAB (minimización de la perturbación) y Square Attack (un ataque *black-box* que no usa gradientes y detecta *gradient masking*). Una muestra se considera robusta solo si resiste los cuatro, lo que lo convierte en el ataque más fuerte de la progresión $\text{FGSM} < \text{PGD} < \text{CW} \lesssim \text{AUTOATTACK}$.

4.6 Arquitecturas

ResNet-18 estándar. Adaptamos la ResNet-18 de `torchvision` a CIFAR (kernel inicial 3×3 , sin maxpool inicial; $\sim 11.2\text{M}$ parámetros). La normalización por canal se incorpora como primera capa del modelo, de modo que los ataques operan coherentemente en el espacio de píxeles $[0, 1]$. Entrenamos 30 epochs con SGD (lr = 0.1, momentum = 0.9, weight decay = 5×10^{-4}), augmentación estándar (random crop + flip) y *cosine annealing* del learning rate.

SimpleCNN (para transferibilidad). Una red convolucional sencilla de $\sim 667\text{K}$ parámetros (tres bloques conv-BN-ReLU con max-pooling), arquitectónicamente distinta a la ResNet-18, usada como modelo objetivo en el experimento de transferibilidad de la Parte II.

Modelos backdoor (Parte III). ResNet-18 entrenadas desde cero sobre versiones envenenadas de CIFAR-10, con distintos porcentajes de envenenamiento.

4.7 Métricas de evaluación

- **Accuracy limpia:** $\text{Acc}(f_\theta) = \frac{1}{N} \sum_i \mathbf{1}[\hat{y}(x_i) = y_i]$.
- **Accuracy adversaria:** $\text{Acc}_{\text{ADV}}(f_\theta, A, \epsilon) = \frac{1}{N} \sum_i \mathbf{1}[\hat{y}(x_i + A(x_i)) = y_i]$, donde A es el ataque a evaluar.
- **Tasa de éxito del ataque:** $1 - \text{Acc}_{\text{ADV}}$ restringido a ejemplos originalmente bien clasificados.
- **Norma promedio de la perturbación:** $\|\bar{\delta}\|_p$ (relevante para C&W, donde ϵ no es fijo).

4.8 Ataques en tiempo de entrenamiento (Parte III)

Backdoor (BadNets). Se inyecta un trigger t (cuadro blanco 3×3) en una fracción ρ del conjunto de entrenamiento, reasignando esas muestras a una clase objetivo. El modelo se entrena desde cero sobre el dataset envenenado.

Label-flipping. Se voltea la etiqueta de una fracción del conjunto de entrenamiento a una clase incorrecta aleatoria, sin modificar las imágenes.

Clean-label (Poison Frogs). Se perturban imágenes manteniendo su etiqueta correcta, minimizando la distancia en el espacio de features ϕ (penúltima capa) a una imagen objetivo de otra clase, con un término de regularización que preserva la apariencia visual.

5 Resultados

5.1 Modelo base

Tras 30 epochs en CIFAR-10 con SGD + cosine annealing, ResNet-18 alcanza una accuracy limpia de 93.46% en el conjunto de prueba. La curva de aprendizaje muestra convergencia monótona sin overfitting significativo, valor consistente con los reportes de la literatura para esta arquitectura sin augmentación adicional ni regularización avanzada.

5.2 Curva accuracy vs ϵ (FGSM)

La Figura 1 reporta la accuracy de la red bajo FGSM al variar la magnitud de la perturbación ϵ . Los valores numéricos correspondientes se resumen en la Tabla 1.

ϵ (escala 0–255)	Accuracy
0 (limpia)	93.46%
1/255	53.85%
2/255	33.64%
4/255	21.45%
8/255 (estándar)	15.84%
16/255	11.65%

Cuadro 1: Accuracy de ResNet-18 en CIFAR-10 bajo FGSM con distintos valores de ϵ . La columna $\epsilon = 0$ corresponde a evaluación sin ataque.

Tres observaciones merecen destacarse:

1. **Decaimiento abrupto a bajo ϵ .** La pérdida más drástica ocurre entre $\epsilon = 0$ y $\epsilon = 1/255$: casi 40 puntos porcentuales en una sola unidad de pixel. Esto es coherente con la hipótesis lineal de Goodfellow et al.: en alta dimensión ($d = 3 \times 32 \times 32 = 3072$), una perturbación coordenada por coordenada de norma ϵ se acumula linealmente en el producto interno con el gradiente, $\langle \nabla \mathcal{L}, \delta \rangle = \epsilon \|\nabla \mathcal{L}\|_1$, lo que puede ser grande incluso con ϵ minúsculo.

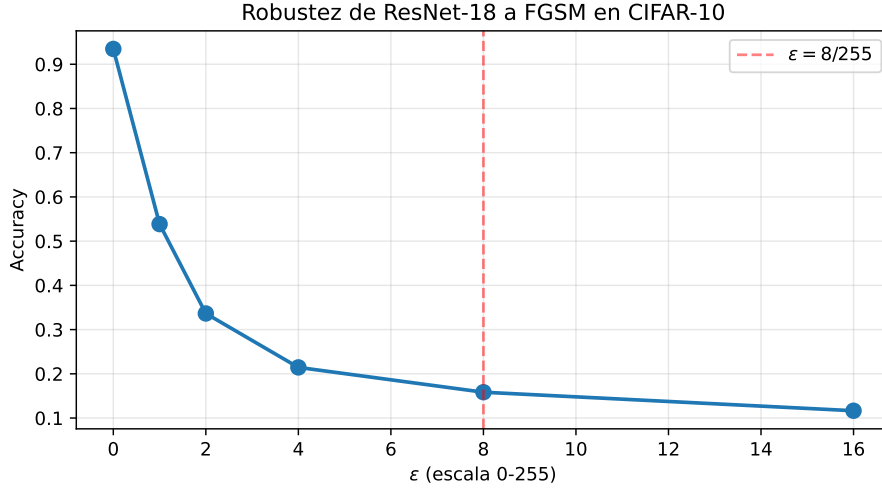


Figura 1: Robustez de ResNet-18 a FGSM en CIFAR-10. La accuracy decae monótonicamente con ϵ : incluso una perturbación de $1/255$ (visualmente imperceptible) reduce la accuracy de 93.46% a 53.85%. En el régimen estándar $\epsilon = 8/255$ (línea roja punteada), la accuracy es 15.84%. A $\epsilon = 16/255$ la accuracy se acerca al nivel de azar ($\sim 11.7\%$, cercano al 10% de 10 clases).

2. **Saturación cerca del nivel de azar.** A $\epsilon = 16/255$, la accuracy de 11.65% se aproxima al azar uniforme sobre 10 clases (10%). La perturbación domina la señal y el modelo predice casi al azar.
3. **El umbral estándar $\epsilon = 8/255$ es realmente severo.** A pesar de ser una perturbación que un humano ni siquiera percibe (cambia el valor de cada píxel en a lo más 3% del rango total), la accuracy cae a 15.84%. Esto justifica la elección de ese umbral en RobustBench como benchmark de la comunidad.

5.3 Comparación de ataques (Parte II)

La Tabla 2 resume la accuracy de la ResNet-18 bajo los tres ataques de evasión. La jerarquía teórica de fortaleza $\text{FGSM} < \text{PGD} \leq \text{CW}$ se confirma de forma contundente.

Modelo	Limpia	FGSM	PGD-40	C&W	AutoAttack
ResNet-18 estándar	93.46	15.84	0.00	0.00	0.00

Cuadro 2: Accuracy (%) limpia y bajo distintos ataques. FGSM y PGD-40 operan en ℓ_∞ con $\epsilon = 8/255$ sobre el test set completo (10 000 imágenes); C&W (en ℓ_2) y AutoAttack (en ℓ_∞ , $\epsilon = 8/255$) operan sobre un subset de 1000 imágenes. AutoAttack, el ensemble estándar de RobustBench, confirma empíricamente que es el ataque más fuerte: sobre un modelo no defendido la accuracy ya es nula desde PGD, por lo que AutoAttack la mantiene en cero.

Verificación de implementaciones. Tanto FGSM como PGD se implementaron desde cero y se compararon contra `torchattacks`. FGSM coincide exactamente (15.84% en ambas); PGD coincide (0.00% en ambas, con diferencias despreciables por el *random start* aleatorio), confirmando la correctitud del código.

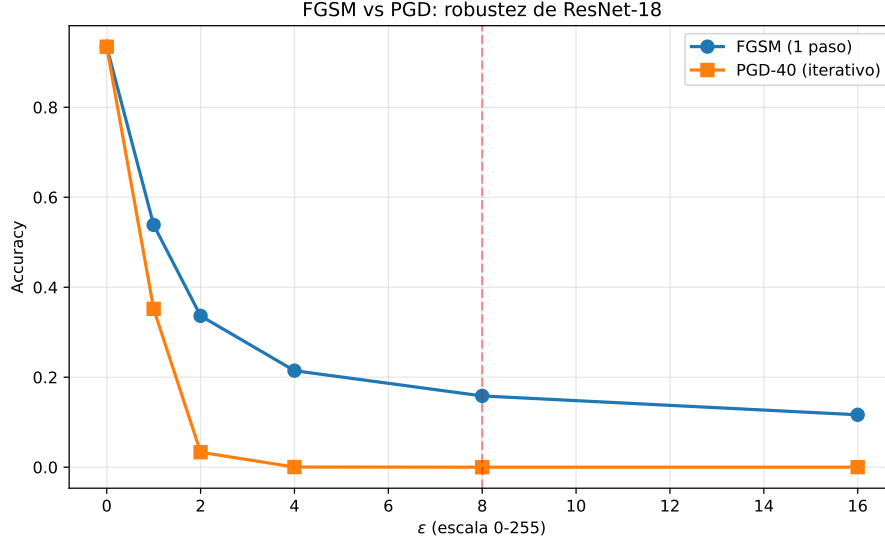


Figura 2: Comparación FGSM vs PGD-40 al variar ϵ . PGD queda siempre por debajo de FGSM (ataque más fuerte). El contraste es dramático a magnitudes bajas: con $\epsilon = 2/255$, PGD ya reduce la accuracy a 3.32% mientras FGSM la deja en 33.64%.

5.4 Análisis de norma de perturbación

La Tabla 3 compara las normas ℓ_2 y ℓ_∞ promedio de las perturbaciones generadas por cada ataque, ilustrando su distinta filosofía.

Ataque	ℓ_2 medio	ℓ_∞ medio
FGSM	1.724	0.0314
PGD-40	1.373	0.0314
C&W	0.187	variable

Cuadro 3: Normas promedio de la perturbación (subset de 1000 imágenes). FGSM y PGD saturan el presupuesto $\ell_\infty = 8/255 \approx 0.0314$, pero PGD logra el ataque con menor energía ℓ_2 (es más “quirúrgico”). C&W alcanza la misma misclassification total con una perturbación ℓ_2 casi diez veces menor que FGSM, reflejando su objetivo de minimizar la perturbación.

5.5 Transferibilidad entre arquitecturas

Para evaluar transferibilidad entrenamos una SimpleCNN (667178 parámetros, accuracy limpia 84.75%), arquitectura distinta a la ResNet-18. Generamos adversarios atacando la ResNet-18 y los evaluamos en ambos modelos (Tabla 4).

La transferibilidad parcial es evidencia a favor de la hipótesis de *non-robust features* [8]: los ejemplos adversarios no son ruido aleatorio, sino que explotan features predictivas pero frágiles que distintas arquitecturas comparten al entrenarse sobre los mismos datos.

5.6 Ejemplos cualitativos

6 Parte III: Ataques en tiempo de entrenamiento

Las Partes I-II cubrieron ataques de *evasión*: el modelo ya está entrenado y el atacante perturba la entrada en inferencia. La Parte III aborda la familia complementaria, en la que el atacante

Adversarios (generados vs ResNet-18)	En ResNet-18 (white-box)	En SimpleCNN (transfer)
FGSM	15.84%	41.53%
PGD	0.00%	40.25%

Cuadro 4: Transferibilidad de adversarios. Los ejemplos generados contra la ResNet-18 también degradan la SimpleCNN (de 84.75% a $\sim 40\%$), pese a que esta nunca fue atacada directamente. En este caso PGD transfiere ligeramente mejor que FGSM (deja la SimpleCNN en 40.25% frente a 41.53%), aunque ambos producen una degradación comparable; la transferibilidad relativa de FGSM y PGD puede variar entre corridas y configuraciones.



Figura 3: Ejemplos adversarios generados con FGSM ($\epsilon = 8/255$) sobre el test set de CIFAR-10. Fila superior: imagen original con la predicción del modelo. Fila central: imagen adversaria, con la nueva predicción (en rojo si es errónea, en verde si el ataque falló). Fila inferior: perturbación δ amplificada y normalizada al rango $[0, 1]$ para visualización. Las perturbaciones son visualmente imperceptibles a simple vista en la imagen adversaria pero suficientes para inducir misclassification en la mayoría de casos.

interfiere con el *proceso de entrenamiento* mismo, comprometiendo el modelo a través de los datos. Estudiamos un ataque de backdoor y dos de envenenamiento.

6.1 Backdoor (BadNets)

Un ataque de backdoor [11] inyecta un *trigger* (un patrón fijo) en una fracción del conjunto de entrenamiento y reasigna esas muestras a una clase objetivo. El modelo aprende simultáneamente la tarea limpia y la asociación trigger $\rightarrow$ objetivo. Formalmente, una muestra envenenada es

$$x_{\text{poison}} = (1 - m) \odot x + m \odot t, \quad y_{\text{poison}} = y_{\text{objetivo}}, \quad (5)$$

donde m es una máscara binaria y t el patrón del trigger. Usamos un cuadro blanco de 3×3 píxeles en la esquina inferior derecha y clase objetivo **airplane**.

Se evalúan dos métricas: la **clean accuracy** (sobre test limpio) y el **attack success rate** (ASR), definido como la fracción de imágenes de test (de clases distintas al objetivo) que, al recibir el trigger, se clasifican como la clase objetivo.

Envenenamiento	Clean accuracy	ASR
0.1% (50 imágenes)	91.72%	73.64%
0.5% (250 imágenes)	91.01%	91.21%
1.0% (500 imágenes)	90.99%	93.69%
5.0% (2500 imágenes)	91.02%	97.27%
10.0% (5000 imágenes)	89.95%	97.31%

Cuadro 5: Backdoor BadNets: attack success rate y clean accuracy en función del porcentaje de envenenamiento. Con tan solo 0.5% del dataset envenenado, el ASR ya supera el 91% mientras la clean accuracy permanece intacta (con 0.1% el ASR es 73.6%, aún sustancial). El modelo entrenado con 5% (experimento dedicado) alcanza 93.20% de clean accuracy —indistinguible de un modelo sano— con un ASR del 97.4%.

6.2 Data poisoning

Label-flipping. El ataque de disponibilidad más simple consiste en voltear las etiquetas de una fracción del conjunto de entrenamiento a clases incorrectas aleatorias. A diferencia del backdoor, este ataque *sí* degrada la accuracy visible (Tabla 6), lo que lo hace ruidoso y detectable.

Etiquetas volteadas	Clean accuracy
0% (baseline)	91.00%
10%	89.55%
20%	88.24%
40%	75.80%

Cuadro 6: Label-flipping: la accuracy decae de forma aproximadamente lineal con el porcentaje de etiquetas corruptas. El contraste con el backdoor es ilustrativo: aquí el daño es visible y proporcional, mientras que en el backdoor es invisible a la validación estándar.

Clean-label poisoning (Poison Frogs). El ataque más sofisticado [12] mantiene las etiquetas *correctas* (un humano que inspeccione el dataset no detecta anomalías) pero perturba las imágenes para colisionar, en el espacio de features ϕ , con una imagen objetivo de otra clase:

$$x_{\text{poison}} = \arg \min_x \|\phi(x) - \phi(x_{\text{target}})\|_2^2 + \beta \|x - x_{\text{base}}\|_2^2. \quad (6)$$

El primer término fuerza la colisión en features; el segundo mantiene la imagen visualmente cercana a la base. En nuestra demostración (usando la penúltima capa de la ResNet-18 como ϕ), la perturbación logró acercar la muestra envenenada un 93% hacia el objetivo en el espacio de features (distancia $5.82 \rightarrow 0.39$) con una perturbación visual moderada ($\ell_2 = 1.02$). Cabe notar que Poison Frogs fue diseñado para escenarios de *fine-tuning* con extractor de features congelado; su efecto en entrenamiento desde cero es considerablemente más débil.

Ataque completo en transfer-learning. Para ejecutar el ataque de extremo a extremo en su escenario natural, simulamos una víctima que toma nuestra ResNet-18 preentrenada, **congela el extractor** ϕ y reentrena solo el clasificador final sobre un conjunto pequeño. Para cada imagen objetivo (de clase **bird**) fabricamos $K = 25$ muestras envenenadas a partir de bases **dog** —con etiqueta **dog** correcta— que colisionan en features con el objetivo, y las inyectamos en el conjunto de *fine-tuning*. El ataque tiene éxito si el objetivo, tras el reentrenamiento, se clasifica como **dog** sin degradar la accuracy global. Sobre 10 objetivos distintos, el ataque logró una **tasa de éxito del 50%** (5/10), manteniendo la clean accuracy prácticamente intacta (90.8% frente a 92.3% sin

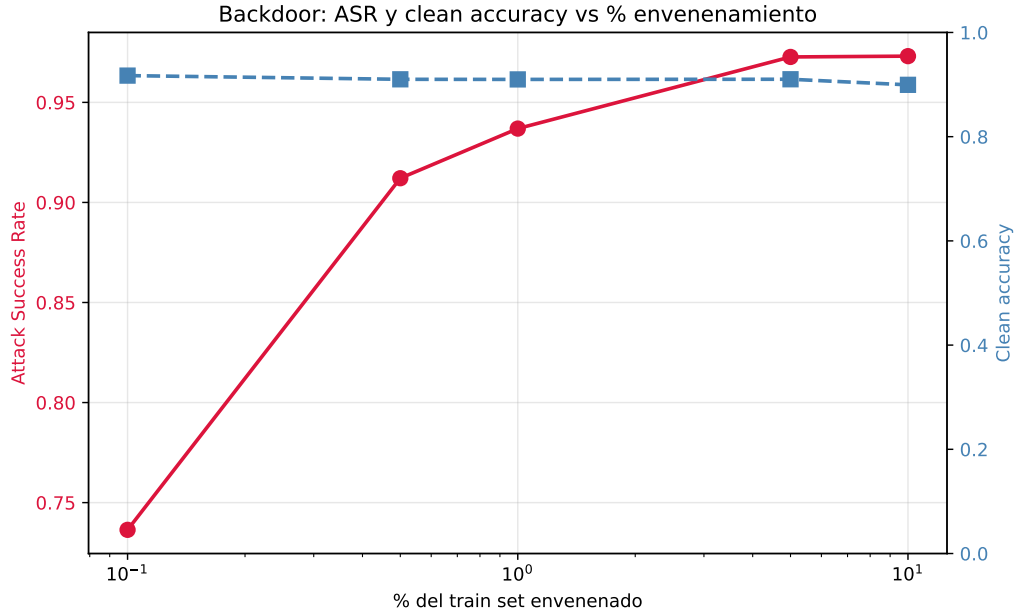


Figura 4: ASR (rojo) y clean accuracy (azul) vs porcentaje de envenenamiento (escala logarítmica). El resultado central de la Parte III: el ASR crece muy rápido —con 0.5% de envenenamiento ya supera el 90%— mientras la clean accuracy se mantiene plana. Esta combinación hace al backdoor prácticamente indetectable mediante validación estándar, pues el modelo “se ve” sano.

ataque). Esto confirma que el clean-label poisoning es viable y sigiloso en su escenario de diseño, aunque notablemente menos confiable que el backdoor: el contraste es ilustrativo —el backdoor es robusto pero requiere etiquetas falsas, mientras que Poison Frogs preserva etiquetas correctas (indetectable por inspección) a costa de una tasa de éxito más variable.

6.3 Comparación de las amenazas en entrenamiento

Ataque	Tipo	Efecto principal
Backdoor (BadNets)	Integridad + trigger	ASR alto, clean accuracy intacta
Label-flipping	Disponibilidad	Degrada clean accuracy global
Clean-label (Frogs)	Integridad dirigida	Colisión en feature space

Cuadro 7: Las tres amenazas en entrenamiento difieren en objetivo y detectabilidad. El backdoor es el más peligroso por su sigilo; el label-flipping el más detectable; el clean-label el más elegante conceptualmente.

7 Conclusiones

Ataques de evasión (Partes I-II).

- Los modelos convolucionales estándar son altamente vulnerables: FGSM con $\epsilon = 8/255$ reduce la accuracy de la ResNet-18 de 93.5% a 15.8%, y PGD-40 la lleva a 0%, confirmando empíricamente la jerarquía $\text{FGSM} < \text{PGD} \leq \text{CW}$.
- C&W logra misclassification total con perturbaciones ℓ_2 casi diez veces menores que FGSM (0.18 vs 1.72), ilustrando el compromiso entre tipo de norma y magnitud de la perturbación.



Figura 5: Backdoor en acción. Fila superior: imágenes limpias correctamente clasificadas. Fila inferior: las mismas imágenes con el trigger de 3×3 píxeles, clasificadas como **airplane** (clase objetivo) pese a su contenido real.



Figura 6: Ataque clean-label (Poison Frogs) en transfer-learning. De izquierda a derecha: imagen *base* (un perro real, etiqueta **dog**); imagen *envenenada* (visualmente sigue siendo un perro y conserva la etiqueta **dog**, pero sus features colisionan con el objetivo); la *perturbación* δ amplificada (de magnitud pequeña, $\ell_2 = 1.55$); y el *objetivo* (un pájaro que, tras inyectar el veneno en el *fine-tuning*, se clasifica como **dog**). La distancia en el espacio de features entre base y objetivo se reduce un 96% ($8.00 \rightarrow 0.32$): la imagen parece un perro pero la red la procesa casi como el pájaro.

- Las implementaciones manuales de FGSM y PGD coinciden con **torchattacks**, validando el código.
- Los adversarios transfieren parcialmente entre arquitecturas (degradan la SimpleCNN de 84.75% a $\sim 40\%$ sin atacarla directamente), evidencia a favor de la hipótesis de *non-robust features*.

Ataques en entrenamiento (Parte III).

- El backdoor es la amenaza más sigilosa: con apenas 0.5% de envenenamiento alcanza un ASR del $\sim 92\%$ manteniendo la clean accuracy intacta, lo que lo hace indetectable por validación estándar.
- El label-flipping degrada la accuracy de forma visible y proporcional, siendo más fácil de detectar y mitigar.
- El clean-label poisoning demuestra que es posible sabotear el entrenamiento manteniendo etiquetas correctas, mediante colisión en el espacio de features. En el escenario de *transfer-*

learning (extractor congelado) el ataque completo logró un 50% de éxito sin degradar la accuracy —sigiloso pero menos confiable que el backdoor.

Síntesis. La vulnerabilidad adversaria es una propiedad estructural de los clasificadores entrenados de forma convencional, manifestándose tanto en inferencia (evasión) como en la cadena de suministro de datos (entrenamiento). Un atacante con acceso a la entrada puede engañar al modelo con perturbaciones imperceptibles; uno con acceso a los datos de entrenamiento puede incrustar comportamientos maliciosos indetectables.

Limitaciones y trabajo futuro. No exploramos defensas: *adversarial training* y certificación (*randomized smoothing*) para evasión; Neural Cleanse [13], *activation clustering* y *spectral signatures* para backdoors; y sanitización de datos para envenenamiento. Tampoco evaluamos contra AutoAttack completo ni ataques físicos. Estas direcciones constituyen la continuación natural del trabajo.

Repositorio

Código disponible en: <https://github.com/jairoandresangelcardenas-tech/dversarial-attacks-cifar10>

Referencias

- [1] C. Szegedy et al. Intriguing properties of neural networks. *ICLR*, 2014.
- [2] I. J. Goodfellow, J. Shlens, C. Szegedy. Explaining and harnessing adversarial examples. *ICLR*, 2015.
- [3] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu. Towards deep learning models resistant to adversarial attacks. *ICLR*, 2018.
- [4] N. Carlini, D. Wagner. Towards evaluating the robustness of neural networks. *IEEE Symposium on Security and Privacy*, 2017.
- [5] F. Croce et al. RobustBench: a standardized adversarial robustness benchmark. *NeurIPS Datasets & Benchmarks Track*, 2021.
- [6] F. Croce, M. Hein. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. *ICML*, 2020.
- [7] H. Kim. Torchattacks: a PyTorch repository for adversarial attacks. *arXiv:2010.01950*, 2020.
- [8] A. Ilyas et al. Adversarial examples are not bugs, they are features. *NeurIPS*, 2019.
- [9] K. Eykholt et al. Robust physical-world attacks on deep learning visual classification. *CVPR*, 2018.
- [10] J. Su, D. V. Vargas, K. Sakurai. One pixel attack for fooling deep neural networks. *IEEE Trans. Evol. Comput.*, 2019.
- [11] T. Gu, B. Dolan-Gavitt, S. Garg. BadNets: identifying vulnerabilities in the machine learning model supply chain. *arXiv:1708.06733*, 2017.
- [12] A. Shafahi et al. Poison Frogs! Targeted clean-label poisoning attacks on neural networks. *NeurIPS*, 2018.
- [13] B. Wang et al. Neural Cleanse: identifying and mitigating backdoor attacks in neural networks. *IEEE Symposium on Security and Privacy*, 2019.
- [14] A. Krizhevsky. Learning multiple layers of features from tiny images. *Tech. Rep.*, University of Toronto, 2009.